# Axona

*Bridges, Relays, and the Programs You Run*

David A. Smith

*Axona.net*

The Axona Services Guide  v4.38.0  2026-07-23

*Axona Services Guide*

*(kernel 4.38.0 · testnet — versioned with the protocol it describes; revised 2026-07-23 with the one-command relay-fleet launcher and the relay update procedure, §3)*

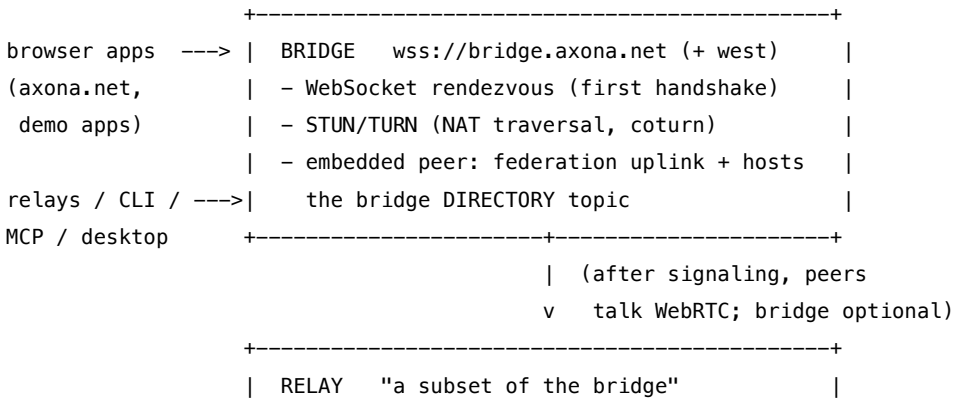
The other programmer-guide documents teach the **library** — how to build your own peer with `@axona/protocol` and speak pub/sub. This guide covers the **services**: the ready-made programs you *run* (or rent) rather than write — the signaling **bridge**, the **relay** and its four front-ends (console, CLI, MCP server, desktop app), the **directory/federation** that ties bridges together, and the **PoW collector**. If you are operating an Axona deployment, wiring an agent into the network, or just want a topic to stay alive when no browser tab is open, this is the document for you.

- **Protocol kernel**: @axona/protocol (v4.38.0)
- **Wire version**: 4.0 (`WIRE_VERSION`); kernel version 4.38.0 (`KERNEL_VERSION`)
- **Live networks**: both run the 4.x line. **Testnet** (`wss://testnet.axona.net`) tracks the newest kernel — 4.38.0, the version this guide describes. The **production** federated pair (`wss://bridge.axona.net` east + `wss://bridge-west.axona.net` west) runs the most recently promoted kernel (4.29.0 at press time), typically one release behind while changes soak on testnet. The two networks are wire-compatible but *separate* — a peer joins one or the other; pick with `RELAY_NETWORK`/`BRIDGE_URL` (§3).
- **Companion docs**:
  - Quick Start · Programmer Guide · API Reference — the library.
  - AI Grounding — building with an AI assistant.
  - Architecture — how the bridge + transport work under the hood.

**Library vs. services.** A *peer* is the unit the library gives you: an identity + a transport + a pub/sub manager, running inside a browser tab, a Node process, or an agent. The library is enough to build a complete app. The **services** exist because some jobs outlive a single tab: brokering the first WebRTC handshake between two strangers (the **bridge**), keeping a topic's messages available when every author has closed their laptop (a **relay** that `host()`s the topic), and letting non-browser callers — shells, agents, GUIs — publish and subscribe (the **CLI / MCP / desktop** front-ends). None of them is privileged: every service is just a peer that has volunteered to stay online and take on a role.

---

## 1. The service map



```
|  a headless peer that joins the mesh to:     |
|  - host()  keyspace / topics  -> durable roots|
|  - publish METRIC snapshots   -> metricTopic  |
|  front-ends over one core (src/ops.js):       |
|  console/TUI . CLI . MCP server . desktop app |
+-----------------------------------------------+
```

| Service | Package / entry | What it is | Runs where |
|---|---|---|---|
| **Bridge** | `axona-bridge` (`src/server.js`) | Signaling rendezvous + STUN/TURN + a federating, directory-hosting peer | A server (Docker stack) |
| **Relay — console/TUI** | `axona-relay` (`src/index.js`, bin `axona-relay`) | Headless node that hosts topics + publishes metrics; live dashboard | A box you keep online |
| **Relay — CLI** | `axona-cli` (`src/cli.js`) | One-shot `pub` / `sub` / `pull` from a shell or script | Anywhere with Node |
| **Relay — MCP server** | `axona-mcp` (`src/mcp.js`) | A **persistent peer** with a durable identity, exposed as native **agent tools** (publish/subscribe/host) over MCP/stdio | Spawned by the agent host |
| **Relay — desktop app** | `axona-relay/desktop` (Electron) | A GUI wrapper around the relay | A laptop/desktop |
| **PoW collector** | `axona-relay/pow-collector.js` (`npm run collect`) | Subscribes to the PoW-bench results topic and archives them | A box you keep online |
| **Directory / federation** | kernel `bridgeDirectory.js` | A *behavior*, not a process: bridges advertise, clients fail over | Inside bridges + clients |

The same `@axona/protocol` kernel runs in every box above. There is no special "server build" — a bridge is a peer that also terminates WebSockets and TURN; a relay is a peer that also `host()`s and publishes metrics.

---

## *2. The Bridge*

### What it is

A **bridge** is the only piece of fixed infrastructure Axona needs, and even it is needed only at the **start** of a connection. Two browsers that have never met cannot exchange WebRTC offer/answer SDP on their own — they need a mutually reachable rendezvous. The bridge is that rendezvous. It does three jobs:

1. **WebSocket signaling** — relays the offer/answer/ICE handshake between two peers until their direct WebRTC data channel is open. After that the bridge is **out of the data path**; peers talk peer-to-peer over the mesh.
2. **STUN/TURN** — a bundled **coturn** gives peers their public reflexive address (STUN) and, for the ~10–15% of NAT pairs that can't connect directly, relays the media (TURN). The bridge mints short-lived TURN credentials in its welcome frame.
3. **An embedded peer** — the bridge process also *joins its own network* as a node. That embedded peer **federates** (an outbound uplink to a sibling bridge, §5) and **hosts the bridge directory topic** so clients can discover other bridges and fail over.

> A bridge is a convenience, not an authority. It cannot read message payloads (they're end-to-end between mesh peers), cannot forge a publish (every envelope is author-signed), and any number of independent bridges can coexist — clients rank and fail over between them.

### Running one

Production bridges run as a **Docker Compose stack** — `bridge` + **Caddy** (automatic TLS) + **coturn** (TURN) — from the `axona-bridge` repo. The one-command path for a fresh Ubuntu/Debian host:

```
curl -fsSL https://raw.githubusercontent.com/axona-net/axona-bridge/main/deploy/install.sh \
  | sudo BRIDGE_DOMAIN=bridge.example.net LETSENCRYPT_EMAIL=you@example.net bash
```

or, with the repo checked out:

```
cd /opt/axona-bridge
docker compose build        # builds first; the old container keeps serving
docker compose up -d        # fast swap; Caddy keeps its certificate
curl https://bridge.example.net/healthz     # →
 ↪  {"status":"ok","version":…,"kernelVersion":"4.38.0"}
```

Full provisioning options (the installer, the Docker bundle, and a manual walkthrough) are in **`axona-bridge/deploy/INSTALL.md`**.

### Configuration

Bridge config lives in `/etc/axona-bridge.env` (systemd) or the compose `.env`:

| Var | Meaning |
|---|---|
| `DOMAIN` / `PUBLIC_IP` | Public hostname + IP (TLS + TURN advertisement) |
| `TURN_AUTH_SECRET` | Shared secret with coturn's `static-auth-secret`; the bridge mints creds from it |
| `BRIDGE_PUBLIC_URL` | The `wss://` URL this bridge advertises in the directory (e.g. `wss://bridge-west.axona.net`) |
| `BRIDGE_DIRECTORY` | `on` (advertise + host the directory) or `off` (isolated fleet — testnet uses `off`) |
| `BRIDGE_LAT` / `BRIDGE_LNG` / `REGION_LABEL` | The bridge's geo anchor + a human label for its directory entry |
| `MIN_PEER_VERSION` | Lowest wire version admitted; older peers get `UPGRADE_REQUIRED` (WS close 4426) |
| `HOST` | Bind address — production sets `127.0.0.1` so the reverse proxy is the only ingress |
| `HEALTHZ_TOKEN` | Gates the **full** `/healthz` body + `/diag`; unauthenticated `/healthz` returns only `{status,version,kernelVersion}` |

**Health & introspection**

- `GET /healthz` — unauthenticated, returns `{status, version, kernelVersion}`.
- `GET /healthz` with header `X-Healthz-Token: <token>` — the full body: synaptome size, directory `{enabled,url}`, uplink status, peer counts.
- `GET /diag` (token-gated) — deeper diagnostics.

---

*3. The Relay*

**What it is**

A **relay is a subset of the bridge**: the embedded-peer half, without the WebSocket/TURN server. It is a headless node you keep online so the network has durable, well-placed members. Two jobs:

1. **Hosting** — `host()` makes the relay a **root** for a keyspace region (or for named topics) *without subscribing*, so it stores-and-serves a topic's messages even when it doesn't consume them. This is what keeps a topic's backlog answerable after every author closes their tab. By default a relay hosts its region's keyspace (`0x89` for `useast`). Under the 4.x **cohort** model a topic is replicated across the K-closest nodes, so durability no longer rests on one relay — several co-hosting relays converge on the same history via anti-entropy, and any of them can answer a read.

2. **Metrics** — served **on demand** (v4.12.0): when any client subscribes to a topic's derived `metricTopic(T)`, a renewable *metrics-on* lease routes to the topic's root — often a relay, since relays root what they host — and while the lease is fresh that root publishes a signed snapshot (`{current_count, seq, subscribers, bytes}`) every ~20 s, giving subscribers live counts + a rolling ~48 h trend instead of polling. The first snapshot lands ~2–20 s after demand turns on; publishing stops ~70 s after the last metric subscriber leaves. (The pre-4.12 push loop — publish every rooted topic on a timer, watched or not — is retired.) **Both open and owned** topics serve metrics (owned-topic activity counts are public too); `seq` is the dense message counter (total events ever, kills included). Because every co-hosting root publishes its own snapshot, the reader-side `peer.metrics()` aggregates across the cohort (sums subscribers, maxes the rest — see the API Reference).

### Running one

```
cd axona-relay
npm install
npm start          # live dashboard (interactive TTY)
npm run probe      # plain timestamped log lines (RELAY_TUI=0) — for piping / services
```

Run it again in another terminal for a second node — the **first** instance claims the persistent identity (stable nodeId, `identity.<region>.json`); each **additional** instance mints a fresh ephemeral identity in the same region. So you get one well-known node plus as many throwaway nodes as you want.

### Running a fleet (one command)

To run several relays at once — on a fresh machine with nothing installed but Node 20+ and Git — use the cross-platform launcher instead of starting each by hand:

```
git clone -b testnet https://github.com/axona-net/axona-relay.git
cd axona-relay
npm install
node scripts/fleet.mjs --region grizzly --count 10 --network testnet
```

The last line is the whole thing. It takes flags, not environment variables:

| Flag | Meaning |
|---|---|
| `--region <name or code>` | Region every relay in the fleet hosts (e.g. `grizzly` or `0x80`). Default `useast`. |
| `--count <n>` | How many relay processes to launch. Default `1`. |
| `--network <prod or testnet>` | Which bridge to bootstrap from. Default `prod`. |
| `--bridge <wss-url>` | Explicit bridge URL (overrides `--network`). |

Each relay is its own child process with a fresh **ephemeral** transport identity (re-minted every start), so N relays never collide. `Ctrl-C` stops the whole fleet gracefully. It is pure Node — iden-

tical on Windows, macOS, and Linux (no bash, no global environment). On Windows this is the supported way to run a fleet, since the `start-fleet.sh` operator script is bash-only.

By default the launcher shows a **live dashboard** — one fixed table, one row per relay (node, state, peers, mesh, roles, subs), redrawn in place — so you can watch fleet health at a glance instead of a wall of interleaved scrolling logs. Pass `--raw` to stream each relay's raw log lines instead (for debugging a single relay), and `--stagger <ms>` to change the gap between launches (default 600 ms; the spread keeps N simultaneous handshakes from overwhelming a small bridge host).

**Configuration (env)**

| Var | Default | Meaning |
|---|---|---|
| `RELAY_NETWORK` | `prod` | Bootstrap network: `prod` (`bridge.axona.net`) or `testnet` (`testnet.axona.net`). Both run the 4.x line; testnet tracks the newest kernel. Use `testnet` to follow the staging line this guide describes; `prod` for production service. |
| `BRIDGE_URL` | — | Explicit bridge URL; **overrides** `RELAY_NETWORK` |
| `RELAY_REGION` | — | `auto` (detect), a region name (`useast`), or code (`0x89`) — sets the nodeId geo prefix |
| `RELAY_LAT` / `RELAY_LNG` | `37.77`/`-122.42` | Geo prefix by coordinate (if `RELAY_REGION` unset). Default = SF (`uswest`) |
| `RELAY_IDENTITY_PATH` | `./identity.<region>.json` | Persisted keypair (stable nodeId) |
| `RELAY_HOST_KEYSPACE` | `1` | Host the region's whole keyspace (set `0` to host only `RELAY_TOPICS`) |
| `RELAY_TOPICS` | — | Comma-separated topic names to host explicitly |
| `RELAY_METRICS` | `1` | Publish metric snapshots (`0` to disable) |
| `RELAY_METRICS_INTERVAL_MS` | ~20000 | Metric publish cadence (~20 s) |
| `RELAY_TUI` | auto | `1` force dashboard, `0` force plain log |

Bridge precedence: `BRIDGE_URL` › `RELAY_NETWORK` › `prod`. Region precedence: `RELAY_REGION` › `RELAY_LAT`/`RELAY_LNG` › default SF.

### Run as a service (systemd)

```
# /etc/systemd/system/axona-relay.service
[Service]
Environment=RELAY_REGION=useast RELAY_TUI=0
ExecStart=/usr/bin/node /opt/axona-relay/src/index.js
Restart=always
```

> **Operator note — process count.** Each relay forks **one worker child**, so a healthy $N$-node fleet is **2 × N** `node src/index.js` processes (a parent + worker per node). Judge fleet health by the distinct **bridge connections** in the logs, not the raw process count. If the **desktop app** (below) is also running, it carries *its own* embedded relay that respawns if killed — expect one extra.

### Updating a relay

The kernel is **vendored** into the relay checkout (committed under `vendor/axona-protocol/`), so following a new testnet release is just a pull and a restart — no separate kernel install:

```
cd axona-relay
git pull                 # brings the new relay + the vendored kernel it ships
# then restart your relay(s):
#   - fleet launcher:  Ctrl-C, then re-run `node scripts/fleet.mjs …`
#   - systemd service: systemctl restart axona-relay
```

Each relay prints its versions in the startup banner — `axona-relay v0.67.0 [EPHEMERAL] (kernel v4.38.0)` — and the bridge reports the kernel it expects at `/healthz`. Keep them in step: a relay on an older kernel still connects and relays, but only relays on the **current** kernel take part in the newest behaviours (for example, cooperative bridge graduation, which a relay must be on kernel ≥ 4.35.0 to honour — an older relay counts against the bridge's connection cap but is never released to make room for a newcomer).

*(Operators who vendor from a local kernel checkout re-vendor with* `npm run sync:protocol` *instead of* `git pull`*; end users who cloned the repo just pull.)*

---

## *4. Relay front-ends*

The relay's core pub/sub lives in `src/ops.js` (`publish` / `subscribe` / `pull`). The **CLI** uses it directly — one throwaway peer per call. The **MCP server** wraps it in a *persistent* peer (`src/mcp-session.js`) so an agent can be a standing participant rather than a series of one-shot connections. The **desktop app** is the GUI relay.

### 4a. CLI (`axona-cli`)

One-shot pub/sub/pull from a shell or script — no long-lived process:

```
npm run pub  -- --topic us-east/hello --message "hi"          # prints { ok, msgId, … }
npm run sub  -- --topic us-east/hello --seconds 20            # JSON lines for N seconds
npm run pull -- --topic us-east/hello                         # the single latest message
```

Good for cron jobs, CI checks, shell glue, and quick manual probes.

### 4b. MCP server (`axona-mcp`)

`src/mcp.js` is a **Model Context Protocol** server that exposes Axona as **native agent tools**, so Claude Code (and any MCP-speaking agent) gets first-class `axona_*` tools instead of shelling out. It speaks JSON-RPC over **stdio** (stdout = protocol, stderr = logs).

**It holds ONE persistent peer** (`src/mcp-session.js`), opened lazily on the first call and kept connected — so the agent is a continuous participant, not a series of one-shot connections. That peer signs with a **durable identity** (both the node identity and the author keypair persist to `~/.axona/claude-mcp-identity.json`), so the agent keeps the **same Author ID and nodeId across restarts** — a stable on-network identity, not a fresh persona each call.

| Tool | Args | Purpose |
|---|---|---|
| `axona_publish` | `topic`, `message`, `region?` | Publish, signed by the stable author |
| `axona_pull` | `topic`, `region?` | Fetch only the latest message |
| `axona_watch` | `topic`, `region?`, `since?` (`all`\|`latest`\|`live`) | Open a **standing subscription**; arrivals buffer server-side |
| `axona_poll` | `topic?`, `peek?`, `max?`, `wait?`, `timeoutSec?` | Drain the buffer (the agent's "inbox"); `wait:true` **long-polls** until a message lands |
| `axona_unwatch` | `topic`, `region?` | Stop a watch, discard its buffer |
| `axona_host` | `topic`, `region?` | **Root** the topic on the agent's peer (store + serve, no subscribe) |
| `axona_unhost` | `topic`, `region?` | Stop hosting |
| `axona_status` | — | Peer nodeId + Author ID, mesh health, per-watch + hosted lists |
| `axona_subscribe` | `topic`, `region?`, `seconds?` (1–120), `since?` | One-shot listen window (back-compat) |

Region defaults to `useast` (`0x89`); subscribers must use the **same region** as the publisher. The server's code default targets **production**; set `RELAY_NETWORK=testnet` to join the staging network instead. Both run the 4.x line, but they are separate networks — the agent participates in whichever one its peer joined, alongside the live apps on that network.

**Receiving — push vs. poll.** Two paths cover the same arrivals: - **Push** — every message on a watched topic is emitted as an MCP *logging notification* (`sendLoggingMessage`) to the client. This is best-effort: it reaches the MCP client, but a turn-based agent loop won't be interrupted by it. - **Long-poll** — `axona_poll(wait:true, timeoutSec)` blocks **server-side** until a message arrives (or the timeout), returning at push latency. This is the reliable pull side. The standing watch buffers anything that lands between polls, so nothing is lost in the gaps.

The agent becomes a **live participant** by pairing `axona_watch` with a `/loop` that calls `axona_poll(wait:true)` each iteration: watch once, then poll-act-repeat. A message that asks for work is a request to execute and reply; **approval for sensitive or irreversible actions stays on the authenticated session, never the pub/sub channel** (a signed message proves the author key, but the topic is still an external data channel — see the security boundary below).

Register it as a project-scoped MCP server with a `.mcp.json` at your repo root:

```
{
  "mcpServers": {
    "axona": { "command": "node", "args": ["/abs/path/to/axona-relay/src/mcp.js"] }
  }
}
```

Point it at testnet (or a specific bridge) with an `env` block — also `MCP_REGION` (default `useast`), `MCP_AUTHOR_PATH` (identity file), `MCP_BUFFER_CAP` (per-watch ring, default 1000):

```
{
  "mcpServers": {
    "axona": {
      "command": "node",
      "args": ["/abs/path/to/axona-relay/src/mcp.js"],
      "env": { "RELAY_NETWORK": "testnet" }
    }
  }
}
```

*Launching it — a procedure for an AI agent  Follow these steps in order. Each has a check; do not proceed until the check passes. Most "the MCP server won't start" reports are step 3 (not restarting) or step 5 (aborting the slow first call) — read those twice.*

**1. Prerequisites (once per machine).** - Node $\geq$ **20**: `node -v`. (WebCrypto and the native transport need it.) - The `axona-relay` repository present locally, and its dependencies installed: `cd axona-relay && npm install`. This step compiles/fetches the **`node-datachannel`** native WebRTC binary — *without it the peer cannot open a connection and every tool call fails.* The kernel is already vendored in the repo (no separate build). - Sanity-check the server file loads: `node --check src/mcp.js` (exit 0).

**2. Register the server.** The server is one command: `node <abs>/axona-relay/src/mcp.js`. Register it with the agent runtime. For Claude Code:

```
claude mcp add axona -- node /abs/path/to/axona-relay/src/mcp.js
# testnet instead of production:
claude mcp add axona --env RELAY_NETWORK=testnet -- node /abs/path/to/axona-relay/src/mcp.js
```

or a project-scoped `.mcp.json` (the two blocks shown above). **Use an absolute path**, and make sure `node` is on the runtime's `PATH`.

**3. Restart / reconnect the agent.** MCP servers are loaded **at client startup**. After `claude mcp add` (or editing `.mcp.json`) you **must restart or reconnect** the session, and approve the new project-scoped server once when prompted. *Skipping this is the number-one reason the tools "don't appear".*

**4. Confirm the tools are present.** After restart, the tools appear as `mcp__axona__axona_status`, `mcp__axona__axona_publish`, etc. If they are absent, the server didn't load — return to step 2/3 (path wrong, `node` not found, or no restart).

**5. Bring the peer online — call `axona_status` and wait for it.** The persistent peer is created **lazily on the first tool call** and then waits for the mesh to form. **The first call can take several seconds, and up to ~30 s on a cold network — this is normal; let it return, do not treat the delay as a failure and do not spam retries** (there is one server process; retrying can't speed it up and only queues more work). **Definition of done:** `axona_status` returns an object with a `nodeId`, an `authorId`, and mesh health showing **at least one peer** (`mesh.peers` $\geq 1$ / synaptome $\geq 1$). That object *is* the proof the server launched and joined the network. If it returns with **zero** peers, the mesh is still forming — wait a few seconds and call `axona_status` again.

**6. Prove a round-trip (optional but recommended).** `axona_publish` a message to a scratch topic, then `axona_pull` the same topic + region — you should read your own message back. Publisher and reader **must use the same `region`** (default `useast`) or they derive different topic IDs and never meet.

**Failure modes and fixes:**

| Symptom | Cause | Fix |
|---|---|---|
| Tools never appear | server not loaded | restart the agent after `claude mcp add`; verify absolute path + `node` on PATH |
| First call hangs ~30 s then errors | mesh can't form (bridge unreachable) | check connectivity; `curl https://bridge.axona.net/healthz` (or `testnet.axona.net`) should return `{"status":"ok"}`; then retry |
| `TransportError: bridge socket closed before open` | transient bridge overload / admission race | **not fatal** — wait a few seconds and call again |
| Cannot find module `node-datachannel` (or similar native error) | `npm install` not run | run `npm install` in `axona-relay` |

| Symptom | Cause | Fix |
|---|---|---|
| `axona_status` shows a `nodeId` a *second* running agent also reports | two MCP processes share the durable identity file (`~/.axona/claude-mcp-identity.json`) → two peers with one nodeId collide on the network | give each agent its own identity: set `MCP_AUTHOR_PATH` to a distinct path per agent/project |
| Published messages never arrive | region or network mismatch | both sides use the same `region`; both join the same network (`RELAY_NETWORK` prod vs testnet) |

**Environment knobs** (set in the `env` block or the `--env` flag): `RELAY_NETWORK` (`prod` default / `testnet`) or `BRIDGE_URL` (explicit, wins); `MCP_REGION` (default `useast`); `MCP_AUTHOR_PATH` (identity file — override for concurrent agents); `MCP_BUFFER_CAP` (per-watch ring, default 1000).

**One-line environment self-test (no agent needed).** Before wiring the MCP server, you can prove the machine can reach the network with the CLI — same core (`src/ops.js`), same failure surface:

```
node src/cli.js pull "axona:bridge-directory"   # connects, returns within ~30 s
```

If that returns JSON, the MCP server will connect too. If it hangs or errors, fix connectivity first (steps 1 and the bridge `/healthz` check above) — the MCP server is not the problem.

> **Security boundary.** The MCP peer is a full publisher/subscriber/host with a durable identity — treat it as a real network participant. Requests and reversible work can flow over a topic, but the agent must keep the approval gate for destructive, irreversible, or outward-facing actions on its authenticated session, not on an Axona message: a signed envelope authenticates *who*, but an open-topic message is still untrusted external input.

### 4c. Desktop app (`axona-relay/desktop`)

An **Electron** GUI wrapper around the relay — the same hosting + metrics node, with a window instead of a terminal. Useful for non-operators who want to contribute a durable node by leaving an app open. Because it embeds a relay, a running desktop app adds one node to your local fleet (and respawns it on crash).

---

## *5. Directory & federation*

A single bridge is a single point of failure. Axona avoids that with two cooperating behaviors, both built into the bridge's embedded peer:

- **Directory.** Each bridge with `BRIDGE_DIRECTORY=on` publishes a signed entry (`{url, lat, lng, label, ver, ts}`) to the well-known open topic `axona:bridge-directory` (pinned to the `useast` region so everyone derives the same Topic ID) and `host()`s that topic so the entry survives. At

launch a client collects the directory, ranks candidates (configured roots → known-good by reputation → fresh from the directory), and **fails over** to a saved alternate if its primary is unreachable.

- **Federation.** A bridge with the directory on also **uplinks** — an outbound `webTransport` connection *into* a sibling bridge — so the two bridges share one connectome. A client connected to **either** bridge can then discover and reach peers on **both**. In the live network, `bridge-west` uplinks to `bridge.axona.net`; the east bridge stays uplink-less as the seed.

  Verify federation against a **warm** topic (both bridges host + republish `axona:bridge-directory`), never a cold fresh topic — a cold topic has no roots yet and reads as a false zero.

---

### *6. The PoW collector*

`pow-collector.js` (`npm run collect`) is a small standing service that subscribes to the proof-of-work benchmark **results** topic and archives each published result for later analysis. It's optional infrastructure for the PoW research line, not part of the core pub/sub path. (It can wedge silently — alive but not receiving — so a restart with `since:'all'` backfills the retained backlog.)

---

### *7. Choosing what to run*

| You want to… | Run |
|---|---|
| Let strangers connect to your app's network | A **bridge** (or use a live one — `bridge.axona.net` for production, `testnet.axona.net` for staging) |
| Keep a topic's messages alive when no author is online | A **relay** that `host()`s it |
| Publish live subscriber counts for a topic | Nothing extra — **any root serves metrics on demand** when a client subscribes to `metricTopic(T)`; a relay just makes the root durable |
| Publish/subscribe from a shell, cron, or CI | The **CLI** (`axona-cli`) |
| Give an AI agent native Axona tools | The **MCP server** (`axona-mcp`) |
| Contribute a node from a desktop, no terminal | The **desktop app** |
| Survive a bridge outage | Two **federated bridges** + the directory |

Most application developers run **none** of these in development — they point the library at the live `bridge.axona.net` and let the existing relay fleet host their topics. You reach for self-hosted services when you want an isolated network, guaranteed topic durability, or an agent/automation integration.

---

*8. Are there analogous systems?*

If you know other realtime/decentralized stacks, the Axona services map onto patterns you've already seen:

| Axona service | Closest analogue elsewhere |
|---|---|
| Bridge (signaling + TURN) | WebRTC **STUN/TURN** servers; **libp2p** circuit-relay v2 / rendezvous |
| Relay (durable topic host) | An **MQTT broker** / **NATS** server with retained messages — but decentralized and unprivileged |
| Bridge directory | **DNS seeds / bootstrap node lists** (Bitcoin, IPFS) |
| MCP server | Any other **MCP integration** (GitHub, Slack) — a thin agent-facing adapter over the library |

The difference in every row is the same: in those systems the server is an *authority* (it owns the namespace, the messages, or the membership). In Axona the service is a *volunteer* — a peer that took on a role and can be replaced by any other peer that takes on the same role.

---

*Where to go next*

- **Programmer Guide** — build the peer that talks to these services.
- **API Reference** — `host()` / `unhost()`, `metricTopic()`, and the rest of the public surface.
- **`axona-bridge/deploy/INSTALL.md`** — provision a bridge (installer, Docker, or manual).
- **`axona-relay/README.md`** — the relay's full configuration + dashboard reference.

Found a gap in this guide? Open an issue at https://github.com/axona-net/axona-docs.